

API

GA7-220501096-AA5-EV02

Camilo Andrés Tibatá Salguero
Aprendiz

DIEGO ARMANDO BARRERA ARAUJO
Instructor

Complejo tecnológico minero agroempresarial

Regional Antioquia
Puerto Berrio

Análisis y desarrollo de software

Ficha

3235874

Introducción

En el desarrollo de aplicaciones web es fundamental comprobar que las diferentes funcionalidades de una API trabajen correctamente y que exista una comunicación adecuada entre el sistema, la base de datos y el usuario.

Para esta actividad se realizó el proceso de testing de la API construida en la evidencia anterior AA5-EV01, utilizando la herramienta Postman.

El objetivo principal de esta evidencia es comprobar el funcionamiento de los diferentes ENDPOINT de la API mediante solicitudes HTTP, verificando las respuestas obtenidas y validando que la información sea procesada y entregada correctamente.

Durante el desarrollo se realizaron pruebas de los ENDPOINT implementados, se revisó la estructura del proyecto, la base de datos creada y el código utilizado para procesar las solicitudes. Finalmente, se documentaron las pruebas realizadas mediante capturas de pantalla y se verificó el funcionamiento de la API mediante Postman.

Desarrollo

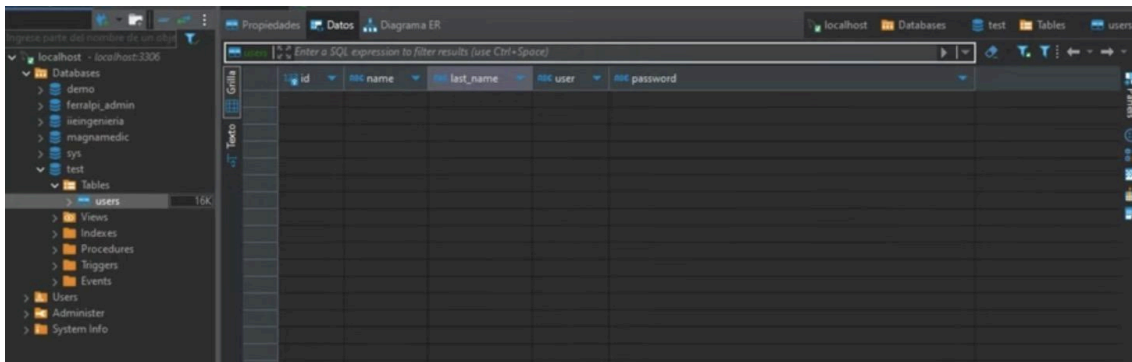
OBJETIVO

Realizar el testing de la API desarrollada en la evidencia AA5-EV01 mediante la herramienta Postman, comprobando el funcionamiento de sus ENDPOINT, la comunicación con la base de datos y las respuestas generadas por el sistema.

1. Base de datos

Como primer paso se realizó la creación y configuración de la base de datos utilizada por el proyecto. Esta base de datos permite almacenar la información necesaria para que la API pueda realizar operaciones de consulta, registro, actualización o eliminación de datos, dependiendo de los ENDPOINT implementados.

La conexión entre la aplicación y la base de datos permite que las solicitudes realizadas mediante la API puedan procesar información almacenada de manera persistente.



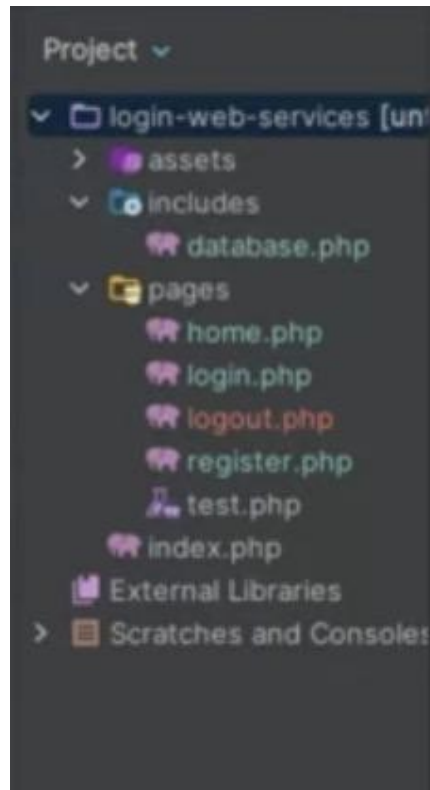
Base de datos creada.

En este pantallazo se evidencia la base de datos utilizada en el proyecto y su estructura correspondiente.

2. Estructura del proyecto

El proyecto fue organizado de manera que cada componente tuviera una función específica. La estructura permite separar la lógica de la aplicación, la conexión con la base de datos, los recursos necesarios y los archivos encargados de recibir y procesar las solicitudes realizadas a la API.

Esta organización facilita el mantenimiento del proyecto y permite identificar con mayor facilidad cada una de las partes que intervienen en el funcionamiento de la API.



En esta captura se evidencia la organización de las carpetas y archivos que conforman el proyecto.

3. Explicación del código

El código desarrollado contiene la lógica necesaria para que la API pueda recibir solicitudes, procesar la información y generar una respuesta para el cliente. Para esto se utilizan diferentes archivos y clases que permiten organizar las funciones del sistema.

La estructura del código facilita la comunicación entre los ENDPOINT, la lógica de procesamiento y la base de datos.

```
login.php database.php x register.php home.php index.php
1 <?php
2 // Archivo: database.php
3
4 // Crear conexión
5 $o = mysqli_connect( hostname: 'localhost', username: 'root', password: '', database: 'test');
6 |
```

```
login.php database.php register.php x home.php index.php
1 <?php
2 session_start();
3
4 // Incluir el archivo de conexión a la base de datos
5 include '../includes/database.php';
6
7 // Definir variables para almacenar los mensajes de error y éxito
8 $messageError = '';
9 $messageSuccess = '';
10
11 // Verificar si se ha enviado una solicitud POST
12 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
13     // Recuperar los datos enviados en formato JSON
14     $json = file_get_contents( filename: 'php://input');
15     $data = json_decode($json, associative: true);
16
17     // Verificar si los datos están completos
18     if (!isset($data['name']) || !isset($data['last_name']) || !isset($data['user']) || !isset($data['password']))
19         $messageError = 'Todos los campos son obligatorios.';
20     else {
21         // Obtener los datos del JSON
22         $name = $data['name'];
23         $last_name = $data['last_name'];
24         $user = $data['user'];
25         $password = $data['password'];
26
27         // Verificar si el usuario ya existe en la base de datos
28         $sql = "SELECT * FROM users WHERE user=?";
29         $stmt = $db->prepare($sql);
30         $stmt->bind_param( types: "s", &$var1: $user);
31         $stmt->execute();
32         $resultado = $stmt->get_result();
33
34         if ($resultado->num_rows > 0) {
35             $messageError = 'El usuario ya existe.';
36         } else {
37             // Hash de la contraseña
38             $hashedPassword = password_hash($password, algo: PASSWORD_DEFAULT);
39         }
40     }
41 }
```

```
login.php database.php register.php x home.php index.php
21 // Obtener los datos del JSON
22 $name = $data['name'];
23 $last_name = $data['last_name'];
24 $user = $data['user'];
25 $password = $data['password'];
26
27 // Verificar si el usuario ya existe en la base de datos
28 $sql = "SELECT * FROM users WHERE user=?";
29 $stmt = $db->prepare($sql);
30 $stmt->bind_param("s", &$user);
31 $stmt->execute();
32 $resultado = $stmt->get_result();
33
34 if ($resultado->num_rows > 0) {
35     $messageError = 'El usuario ya existe.';
36 } else {
37     // Hash de la contraseña
38     $hashedPassword = password_hash($password, PASSWORD_DEFAULT);
39
40     // Insertar el nuevo usuario en la base de datos
41     $sql = "INSERT INTO users (name, last_name, user, password) VALUES (?, ?, ?, ?)";
42     $stmt = $db->prepare($sql);
43     $stmt->bind_param("ssss", &$name, &$last_name, &$user, &$hashedPassword);
44
45     if ($stmt->execute()) {
46         // Si el usuario se registra correctamente, mostrar un mensaje de éxito
47         $messageSuccess = 'Usuario creado exitosamente.';
48         echo json_encode(array("success" => "Usuario creado exitosamente."));
49         exit;
50     } else {
51         $messageError = 'Error al registrar el usuario.';
52         echo json_encode(array("error" => "Error al registrar el usuario."));
53         exit;
54     }
55 }
56 }
57 }
58 ?>
```

```
login.php x database.php register.php home.php index.php
1 <?php
2 session_start();
3
4 // Incluir el archivo de conexión a la base de datos
5 include '../includes/database.php';
6
7 // Definir una variable para almacenar el mensaje de error
8 $messageError = '';
9
10 // Verificar si se ha enviado una solicitud POST para iniciar sesión
11 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
12     // Recuperar los datos enviados en formato JSON
13     $json = file_get_contents('php://input');
14     $data = json_decode($json, associative: true);
15
16     // Verificar si los datos están completos
17     if (!isset($data['user']) || !isset($data['password'])) {
18         $messageError = 'El usuario y la contraseña son obligatorios.';
19     } else {
20         // Obtener los datos del JSON
21         $user = $data['user'];
22         $password = $data['password'];
23
24         // Consultar la base de datos para verificar las credenciales del usuario
25         $sql = "SELECT * FROM users WHERE user=?";
26         $stmt = $db->prepare($sql);
27         $stmt->bind_param('s', $user);
28         $stmt->execute();
29         $resultado = $stmt->get_result();
30
31         // Verificar si se encontró un usuario con el nombre proporcionado
32         if ($resultado->num_rows == 1) {
33             // Obtener el hash de la contraseña almacenado en la base de datos
34             $fila = $resultado->fetch_assoc();
35             $hashAlmacenado = $fila['password'];
36
37             // Verificar si la contraseña ingresada coincide con el hash almacenado
38             if (password_verify($password, $hashAlmacenado)) {
39                 // Autenticación exitosa, establecer la sesión del usuario
40             }
41         }
42     }
43 }
```

```
login.php x database.php register.php home.php index.php
21 $user = $data['user'];
22 $password = $data['password'];
23
24 // Consultar la base de datos para verificar las credenciales del usuario
25 $sql = "SELECT * FROM users WHERE user=?";
26 $stmt = $db->prepare($sql);
27 $stmt->bind_param("s", &$user);
28 $stmt->execute();
29 $resultado = $stmt->get_result();
30
31 // Verificar si se encontró un usuario con el nombre proporcionado
32 if ($resultado->num_rows == 1) {
33     // Obtener el hash de la contraseña almacenado en la base de datos
34     $fila = $resultado->fetch_assoc();
35     $hashAlmacenado = $fila['password'];
36
37     // Verificar si la contraseña ingresada coincide con el hash almacenado
38     if (password_verify($password, $hashAlmacenado)) {
39         // Autenticación exitosa, establecer la sesión del usuario
40         $_SESSION['usuario'] = $user;
41         // Respondemos con un mensaje de éxito en formato JSON
42         echo json_encode(array("success" => "Inicio de sesión exitoso"));
43         exit;
44     } else {
45         // Credenciales incorrectas, responder con un mensaje de error en formato JSON
46         echo json_encode(array("error" => "Credenciales incorrectas"));
47         exit;
48     }
49 } else {
50     // Credenciales incorrectas, responder con un mensaje de error en formato JSON
51     echo json_encode(array("error" => "Credenciales incorrectas"));
52     exit;
53 }
54 }
55 }
56 ?>
57
58 <!DOCTYPE html>
```

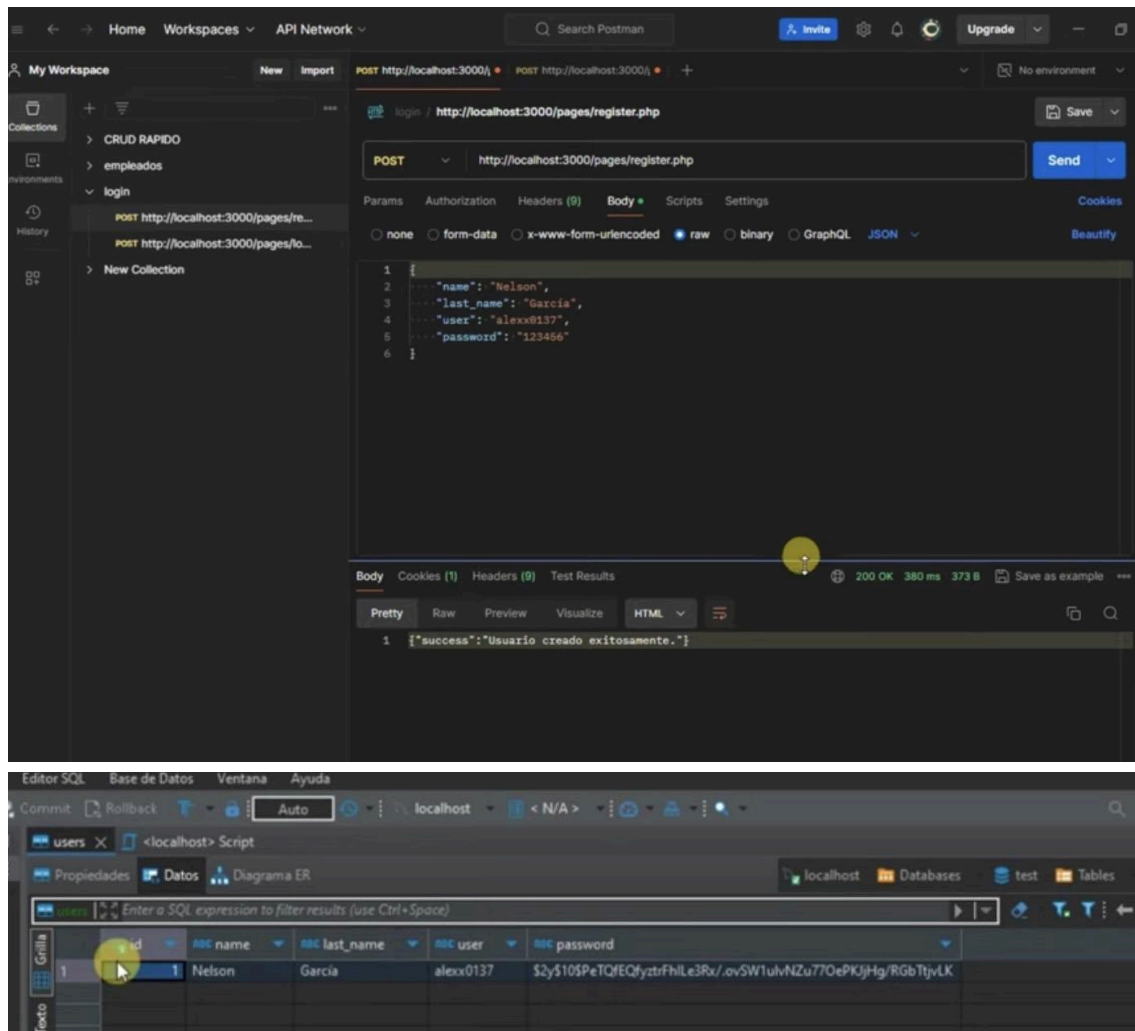
Código del proyecto.

En esta captura se muestra una de las partes principales del código utilizada para el funcionamiento de la API, Código relacionado con la conexión o procesamiento de datos, En este pantallazo se evidencia el código encargado de establecer la comunicación necesaria para trabajar con los datos de la aplicación, Código de la lógica de la API, Esta captura muestra la lógica utilizada para procesar las solicitudes realizadas a los diferentes ENDPOINT, Código de respuesta de la API, En esta parte se evidencia la generación y entrega de la respuesta producida por la API después de procesar una solicitud, Código correspondiente a otro componente de la API, En este pantallazo se muestra otra sección del código que participa en el funcionamiento general del proyecto.

Pruebas de la API utilizando Postman

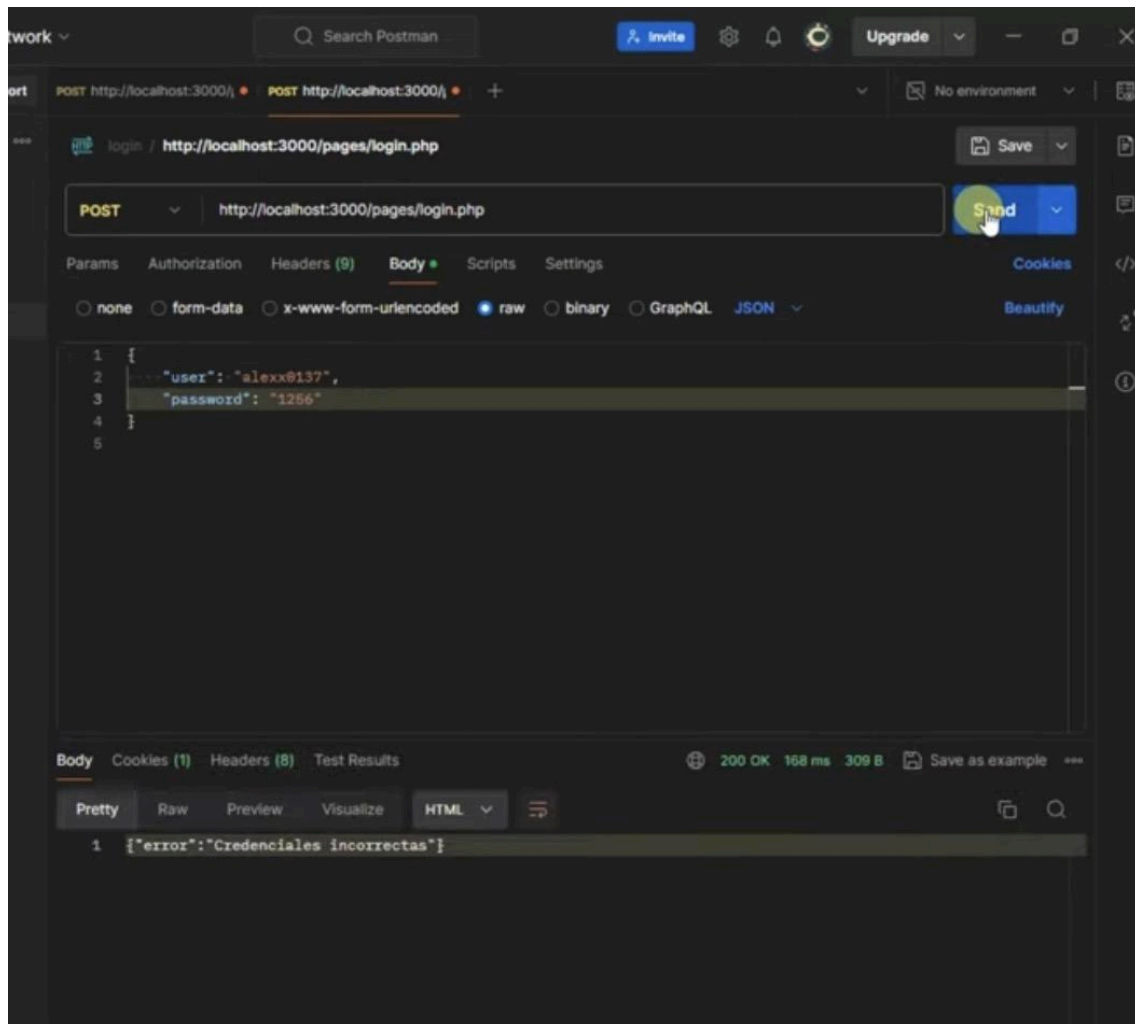
Primera prueba realizada en Postman.

En esta prueba se realizó una solicitud al primer ENDPOINT de la API. Se verificó que el servidor recibiera correctamente la solicitud y generara una respuesta.



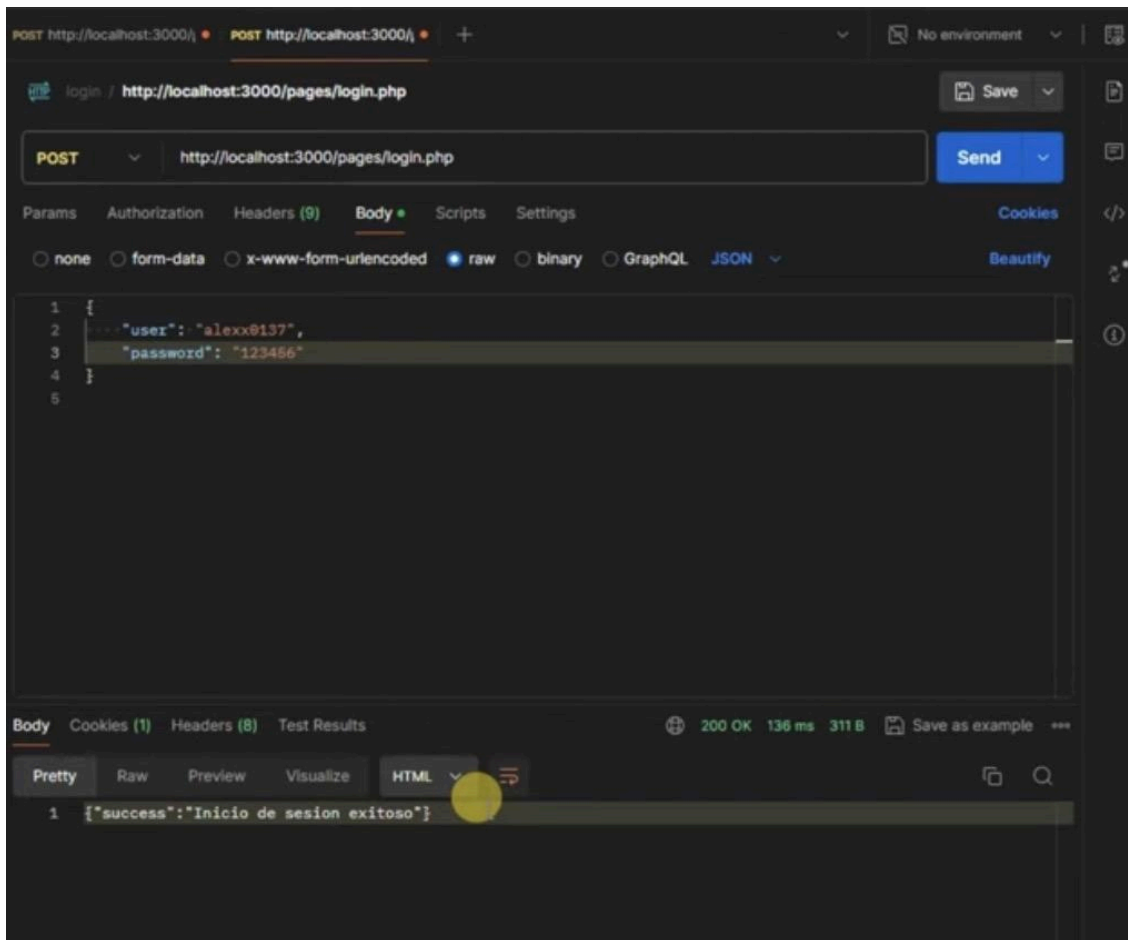
Segunda prueba realizada en Postman.

En esta captura se evidencia otra solicitud realizada mediante Postman, permitiendo comprobar el funcionamiento de otro ENDPOINT de la API.



Tercera prueba realizada en Postman.

En esta prueba se verificó el procesamiento de la solicitud y la información entregada como respuesta por parte de la API.



El funcionamiento de una API puede comprenderse mediante el ciclo de vida del dato. Este proceso representa el recorrido que realiza la información desde que es almacenada hasta que finalmente es presentada al usuario.

1. Almacena – SQL

El primer paso corresponde al almacenamiento de la información en una base de datos. Mediante SQL y las operaciones DML, como INSERT, SELECT, UPDATE y DELETE, se pueden crear, consultar, modificar y eliminar los datos necesarios para el funcionamiento de la aplicación.

2. Procesa – PHP

Después de obtener los datos, PHP se encarga de procesarlos mediante la lógica definida en el proyecto. Las clases y funciones permiten organizar las operaciones y establecer las reglas que debe seguir la aplicación.

3. Empaqueta – JSON/XML

Una vez procesada la información, esta debe ser preparada para poder ser enviada. Para esto se utilizan formatos de intercambio como JSON o XML. En una API REST es común utilizar JSON debido a su facilidad de lectura y procesamiento.

4. Transporta – Reglas SOAP/REST

La información es transportada mediante protocolos y reglas de comunicación. En el caso de una API REST se utilizan solicitudes HTTP y métodos como GET, POST, PUT y DELETE para realizar las diferentes operaciones.

5. Verifica – Testing

El funcionamiento de la API debe ser comprobado mediante pruebas. En esta evidencia se utiliza Postman para enviar solicitudes a los diferentes ENDPOINT y verificar las respuestas generadas por el servidor.

Aunque el enunciado general menciona SOAP UI como herramienta de testeo, en esta evidencia el proceso solicitado específicamente es el testing mediante Postman, por lo cual las pruebas documentadas corresponden a esta herramienta.

6. Renderiza – Frontend interactivo

Finalmente, la información procesada por la API puede ser utilizada por el frontend para mostrar los resultados al usuario. De esta manera, el usuario interactúa con una interfaz sin necesidad de conocer todos los procesos internos que ocurren detrás de ella.

La integración de estas etapas permite convertir la complejidad de múltiples tecnologías en una experiencia de usuario continua e invisible. El usuario realiza una acción desde el frontend, la solicitud llega a la API, esta procesa la información, consulta o modifica la base de datos y devuelve una respuesta que puede ser mostrada nuevamente en la interfaz.

7. Resultado general de las pruebas

Las pruebas realizadas mediante Postman permitieron verificar el funcionamiento de los ENDPOINT desarrollados. Se comprobó la comunicación entre el cliente y el servidor y se observaron las respuestas generadas por la API.

El uso de Postman facilitó la identificación de posibles errores y permitió comprobar de manera independiente cada una de las operaciones implementadas, antes de que estas sean utilizadas directamente desde el frontend de la aplicación.

Conclusión

- La realización de esta actividad permitió comprobar la importancia del testing dentro del desarrollo de una API. Mediante Postman fue posible realizar solicitudes directamente a los ENDPOINT y verificar las respuestas obtenidas, permitiendo validar el funcionamiento de las operaciones implementadas.
- También se pudo evidenciar cómo diferentes tecnologías trabajan conjuntamente dentro del ciclo de vida del dato. La información parte desde una base de datos SQL, es procesada mediante PHP, empaquetada en un formato como JSON, transportada mediante HTTP y finalmente utilizada por el frontend para presentar una respuesta al usuario.
- El proceso de pruebas permite aumentar la confiabilidad de la aplicación, ya que facilita detectar errores en los ENDPOINT y comprobar que la comunicación entre los diferentes componentes del proyecto se realice correctamente.
- De esta manera, la integración entre base de datos, backend, API, herramientas de testing y frontend permite construir una aplicación funcional en la que los diferentes procesos trabajan de forma coordinada para ofrecer una experiencia continua al usuario.